1.0 Introduction To Computer High Level Language

Definition: High-level programming languages are designed to be more human-readable and easier to use compared to low-level languages like machine code or assembly. They provide higher levels of abstraction, allowing programmers to focus on solving problems rather than dealing with the underlying hardware details.

Some key features of high-level languages include:

i. Use of English-like syntax and keywords that are easier for humans to understand

ii. Support for data types, control structures, and modular programming concepts

iii. Availability of standard libraries for common tasks like file I/O and networking

iv. Portability across different hardware and operating systems

High-level languages can be classified as either compiled (e.g. C, C++, Java) or interpreted (e.g. Python, JavaScript, Ruby). Compiled languages are translated to machine code before execution, while interpreted languages are executed directly by an interpreter.

1.2 The advantages of high-level languages

i. **Abstraction and Simplification:** High-level languages provide a higher level of abstraction, allowing programmers to focus on the logic and functionality of their programs rather than the intricate details of hardware or low-level operations.

ii. **Readability and Maintainability:** Code written in high-level languages is often more readable and understandable, making it easier for programmers to collaborate, debug, and maintain software projects over time.

iii. **Productivity:** High-level languages offer built-in functions, libraries, and frameworks that expedite the development process, boosting productivity and enabling faster creation of complex applications.

iv. **Portability:** Programs written in high-level languages are generally more portable, as they are not tightly bound to a specific hardware architecture, allowing code to be executed on different platforms with minimal modifications.

v. **Reduced Errors:** The abstraction and automation provided by high-level languages reduce the likelihood of human errors, such as memory management issues, that are common in low-level languages.

vi. **Rapid Development:** High-level languages often provide features like dynamic typing, automatic memory management, and concise syntax, enabling rapid prototyping and development of software applications.

vii. **Community and Resources:** Popular high-level languages have large and active communities, resulting in extensive documentation, tutorials, and online resources that aid programmers in learning and problem-solving.

1.3 The main differences between high-level and low-level languages are:

S/N	High-Level Language	Low-Level Language
1.	Easy to learn, understand, and maintain	Difficult for humans to understand but easy for machines
2.	Less memory efficient, consuming more memory	High memory efficiency, consuming less energy

3.	Portable across different platforms	Non-portable and machine-dependent
4.	Easier to debug	Complex to debug
5.	Widely used for programming	Less commonly used nowadays
6.	Takes more time for execution due to translation	Translation speed is high

1.4 Flowchart and Pseudo Code

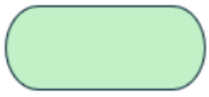
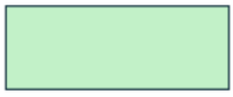
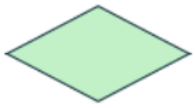
Flowchart

A flowchart is a visual representation of a process or algorithm using a combination of shapes and arrows. It is a tool used to illustrate the steps involved in a program or process, making it easier to understand and communicate the logic of the algorithm. Flowcharts are commonly used in various fields, including programming, education, business, and engineering.

Pseudocode

Pseudocode is a high-level description of a computer algorithm or program using a combination of natural language and programming language elements. It is used to outline the key principles of an algorithm without the need for actual coding syntax. Pseudocode is often used in textbooks, scientific publications, and during the planning phase of program development.

1.5 Flowchart Symbols

Symbol	Name	Function
	Oval	Represents the start or end of a process
	Rectangle	Denotes a process or operation step
	Arrow	Indicates the flow between steps
	Diamond	Signifies a point requiring a yes/no
	Parallelogram	Used for input or output operations

Example: Draw a flowchart to calculate area of circle

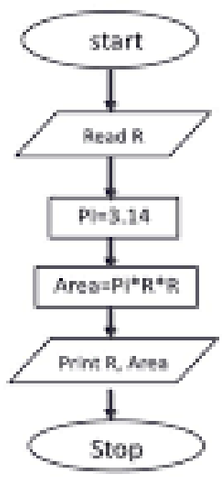
Solution.

We shall write this algorithm in pseudocode. From that, we can do the flowchart.

1. Start

2. Read value of R
3. Set Pi equals to 3.14
4. Calculate Area = pi*R*R
5. Print Area
6. Stop

Flowchart



2.0 Introduction to Objects Oriented Programming

Definition: Object-oriented programming (OOP) is a programming paradigm that revolves around the concept of objects and their interactions. It aims to implement real-world entities like inheritance, hiding, polymorphism, etc., in programming. Here are the key concepts and principles of OOP:

Key Concepts of OOP

- Class: A class is a blueprint or a template that defines the properties and behavior of an object. It is a user-defined data type that consists of data members and member functions.
- Object: An object is an instance of a class. It has its own set of attributes (data) and methods (functions) that define its behavior.
- Inheritance: Inheritance is the mechanism by which one class can inherit the properties and behavior of another class. This allows for code reuse and facilitates the creation of a hierarchy of classes.
- Polymorphism: Polymorphism is the ability of an object to take on multiple forms. This can be achieved through method overriding or method overloading.
- Encapsulation: Encapsulation is the concept of bundling data and methods that operate on that data within a single unit, making it harder for other parts of the program to access or modify the data directly.
- Abstraction: Abstraction is the process of exposing only the necessary information to the outside world while hiding the internal details.

2.2 Introduction to C Programming

C programming was developed by Dennis Ritchie at Bell Labs in the 1970s. C is a general-purpose programming language that provides low-level memory management and direct access to hardware resources. C is known for its concise syntax and lack of explicit memory

management. C is widely used for developing operating systems, embedded systems, and other low-level system software.

2.3 Introduction to C++ Programming

C++ was developed by Bjarne Stroustrup at Bell Labs in the 1980s as an extension to the C language. C++ is a general-purpose programming language that supports object-oriented programming, templates, and generic programming. C++ is known for its verbose syntax and explicit memory management. C++ is widely used for developing operating systems, embedded systems, and other low-level system software, as well as for high-level application programming.

Character set of C and C++

The character set in C and C++ refers to the set of valid characters that can be used in a program. Here are the key points about the character set in C and C++:

Character Set in C Program

i. Source Character Set: The source character set includes all the characters that can be used in the source code of a C program. It includes alphabets, digits, special symbols, and white spaces.

ii. Execution Character Set: The execution character set includes all the characters that can be used during the execution of a C program. It includes all the characters in the source character set, plus additional characters that are specific to the execution environment.

iii. ASCII Values: The C character set is based on the ASCII (American Standard Code for Information Interchange) character set, which includes 128 characters.

iv. Character Constants: Characters can be used as constants in C, such as newline characters ('\n') or escape sequences.

v. String Handling: The character set is used extensively in creating and manipulating strings in C.

vi. Input and Output: The character set is used when reading input and displaying output using functions like printf and scanf.

Character Set in C++ program

i. Source Character Set: The source character set in C++ is implementation-defined, but it typically includes all the characters in the ASCII character set, plus additional characters specific to the implementation.

ii. Execution Character Set: The execution character set in C++ is also implementation-defined, but it typically includes all the characters in the source character set, plus additional characters specific to the execution environment.

iii. Unicode Support: C++ supports Unicode characters, which can be specified using universal characters in the form \uXXXX or \UXXXXXXXX.

iv. Character Classification: The character set is used for character classification functions provided by the <ctype.h> library, such as isalpha, isdigit, and islower, to check character properties.

2.4 Keywords (Reserve) Words

C program Keywords

C has 32 reserved keywords such as int, if, while, return, etc.

These keywords have predefined meanings and cannot be used as variable or function names.

C++ program Keywords

C++ has 95 reserved keywords, including the 32 C keywords plus 63 additional keywords.

The additional keywords include class, public, private, virtual, const_cast, etc.

C++ also has 11 alternative operator keywords like and, not, xor for improved readability.

Features of C++ Program

i. Abstract Data Types: C++ supports abstract data types, which are data types that are defined by their behavior rather than their implementation. This allows for more flexibility and modularity in programming.

ii. Input and Output: C++ provides a variety of input and output functions, such as cin and cout, to interact with the user and display output.

iii. Result: The result of a C++ program is the output generated by the program, which can be displayed on the screen or stored in a file.

iv. Inheritance: C++ supports inheritance, which allows a class to inherit the properties and behavior of another class. This enables code reuse and facilitates the creation of a hierarchy of classes.

v. Header Files: C++ uses header files to include declarations and definitions of functions and variables. This allows for modular programming and makes it easier to reuse code.

vi. Late Binding: C++ supports late binding, which means that the binding of a function or variable to its implementation is delayed until runtime. This allows for more flexibility and dynamic behavior in programming.

vii. Inline Specification: C++ supports inline specification, which allows a function to be defined inline within the code where it is used. This can improve performance and reduce memory usage.

viii. Overloading: C++ supports function overloading, which allows multiple functions with the same name to be defined with different parameter lists. This enables more flexibility and expressiveness in programming.

3.0 Classes in C++

3.1 Classes in C++ program

Reference to a Class: A reference to a class is a variable that holds the memory address of an object of that class. It is declared using the ampersand symbol (&) after the class name.

Example:

```
class MyClass {
public:
   int value;
};
MyClass& myRef; // Reference to MyClass
```

Reference to a Class Member: A reference to a class member is a variable that holds the memory address of a member of a class. It is declared using the ampersand symbol (&) after the class member name.

Example:

```
class MyClass {
```

```
public:
    int value;
};
MyClass& myRef = MyClass(); // Reference to MyClass member
```

3.2 Constructor and Destructor

Constructors are used to initialize the object when it is created.

Destructors are used to clean up the resources used by the object when it is destroyed.

Multiple Inheritance

Multiple inheritance is a feature of C++ where a class can inherit properties or behavior from multiple base classes.

Syntax: class DerivedClass: public BaseClass1, public BaseClass2 { ... };

4.0 Object in C++

4.1 Static Object

A static object is a type of object that is allocated memory only once and is shared by all instances of the class.

Syntax: static A obj;

4.2 Difference Between Inheritance and Friendship

Inheritance is a mechanism in which a derived class acquires the properties and behavior of a base class.

Syntax: class DerivedClass : [access_specifier] BaseClass { ... };

Access Specifiers: The access specifiers (public, protected, private) determine the accessibility of inherited members in the derived class.

While

Friendship is a mechanism that allows a non-member function or another class to access the private and protected members of a class.

Syntax: friend return_type function_name(parameters); or friend class ClassName;

Access Specifiers: Friendship grants access to private and protected members, regardless of the access specifier.

Solving Problems

Exercise 1: Print a welcome text in a separate line.

```
#include <iostream>

using namespace std;


int main() {
   cout << "\n\n Print a welcome text in a separate line :\n";
   cout << "-------------------------------------------\n";
```

```
   cout << " Welcome to \n" ;
   cout << " w3resource.com "<<endl ;
   return 0;
}
```

Exercise 2: Print the sum of two numbers.

```
#include <iostream>
using namespace std;
int main() {
   cout << "\n\n Print the sum of two numbers :\n";
   cout << "-----------------------------------\n";
   cout << " The sum of 29 and 30 is : "<< 29+30 <<"\n\n" ;
   return 0;
}
```

Exercise 3: Calculate the area of a circle.

```
#include <iostream>
using namespace std;
int main() {
   double radius = 5.0;
   double pi = 3.14;
   double area = pi * radius * radius;
   cout << "Area of the circle is: " << area << endl;
   return 0;
}
```

5.0 Interpolation

5.1 Definition

Interpolation is a method of estimating the value of a function or data at an intermediate point based on the known values at surrounding points.

Types of Interpolation

i. Linear Interpolation:

Linear interpolation is the simplest form of interpolation, where a straight line is drawn between two known data points to estimate the value at an intermediate point.

The formula for linear interpolation is: y = y0 + ((y1 - y0) / (x1 - x0)) * (x - x0), where (x0, y0) and (x1, y1) are the known data points, and x is the point at which the value needs to be interpolated.

ii. Polynomial Interpolation: uses a polynomial function to fit the known data points and estimate the value at an intermediate point.

The degree of the polynomial depends on the number of known data points.

6.0 Statistical Program

6.1 Definition

Statistical programs are software tools used for statistical analysis, data visualization, and data manipulation.

There are various types of statistical programs, including:

i. General Purpose Programs: These programs are designed to perform a wide range of statistical tasks, such as data analysis, visualization, and modeling. Examples include R, SAS, SPSS, and Stata.

ii. Specialized Programs: These programs are designed for specific tasks or industries, such as medical imaging, finance, or marketing research. Examples include E-views, Stata, and Excel.

iii. Open-Source Programs: These programs are free and open-source, allowing users to modify and customize the code. Examples include R, Python, and Julia.

6.2 Correlation

Correlation is a statistical measure that describes the strength and direction of the linear relationship between two continuous variables.

Moving Averages of a Time Series

Moving averages are a type of time series analysis that involves calculating the average value of a time series over a specified period.

Chi-Square Test of Independence

The chi-square test of independence is a statistical test used to determine whether two variables are independent.

7.0 Statements in C++

7.1 Input Statements:

Definition: The input statement is a programming construct used to allow a user to provide data to a program during its execution.

cin: The standard input stream in C++ used to read input from the keyboard.

Example: cin >> variable;

getline(): Used to read a line of input (including whitespace) and store it in a string variable.

Example: getline(cin, stringVariable);

scanf(): A C-style input function that can read different data types.

7.2 Output Statements:

Definition: The output statement is a programming construct used to display data or results to the user or to an output device like a file or printer.

cout: The standard output stream in C++ used to display output on the console.

Example: cout << "Hello, World!" << endl;

printf(): A C-style output function that can print different data types.

Example: printf("The value is: %d", intVariable);

write(): Used to write binary data to an output stream.

Example: write(buffer, sizeof(buffer));

The search results also provided

8.0 Random Number Generators

8.1 Definition

A random number generator (RNG) is a software or hardware component that generates a sequence of numbers that appear to be random and unpredictable.

Types: There are several types of RNGs, including:

i. Linear Congruential Generators (LCGs): Use a recurrence relation to generate numbers.
ii. Quadratic Congruential Generators (QCGs): Use a quadratic recurrence relation to generate numbers.
iii. Pseudorandom Number Generators (PRNGs): Use a deterministic algorithm to generate numbers that appear to be random.

Properties: RNGs should have the following properties:

i. Unpredictability: The generated numbers should be unpredictable.
ii. Uniformity: The generated numbers should be uniformly distributed.
iii. Independence: The generated numbers should be independent.

8.2 Normal Variates

A normal variate is a random variable that follows a normal distribution.

Normal variates have the following properties:

i. Mean: The mean of a normal variate is the expected value of the distribution.
ii. Variance: The variance of a normal variate is the spread of the distribution.
iii. Symmetry: Normal variates are symmetric around the mean.

Randomness Tests

Randomness tests are used to evaluate the quality of a random number generator.

There are several types of randomness tests, including:

i. Autocorrelation Test: Tests for correlation between consecutive numbers.
ii. Gap Test: Tests for gaps in the sequence of numbers.
iii. Poker Test: Tests for the distribution of numbers in a sequence.

Application of C++ in Graphics

Turbo C++ graphics programming provides a set of functions and libraries for creating basic 2D graphics applications in C++.

Applications that can be created using Turbo C++ graphics programming:

i. Games: Create simple 2D games like Tic-Tac-Toe, Hangman, or a maze game. Use graphics functions to draw the game board, characters, and handle user input.

ii. Animations: Develop animations of moving objects like a bouncing ball or shapes changing over time. Use delay() and cleardevice() functions to create the animation effect.

iii. Data Visualization: Build bar charts, pie charts, or other data visualization tools to graphically represent information. Use functions like bar() and pie() to draw the chart elements.

iv. Paint Program: Create a simple paint application that allows the user to draw lines, shapes, and fill with colors using the mouse. Handle mouse input events to enable the drawing functionality.

v. Digital Clock: Make a clock that displays the current time in a graphics window. Use outtextxy() to print the time and update it every second.

vi. Fractal Patterns: Generate fractal patterns like the Mandelbrot set or Koch snowflake using recursive graphics functions.

vii. Text Editor: Develop a basic text editor with a graphical user interface. Use graphics functions to draw the text area, cursor, and menu options. Handle keyboard input to edit the text.

viii. Calculator: Create a simple calculator application with graphical buttons and display. Use graphics functions to draw the calculator interface and handle user input.

Addressing Screen Coordinates

The graphics screen uses a Cartesian coordinate system with the origin (0,0) at the top-left corner. Positive x is right, positive y is down. Coordinates are specified as (x,y) pairs.

Drawing Points

Use the putpixel() function to plot a single pixel at the specified (x,y) coordinate with the current drawing color.

Example

cpp

```
putpixel(x, y, getcolor());
```

Setting Colors

Use setcolor() to set the current drawing color. getcolor() returns the current drawing color. setbkcolor() sets the background color.

Example

cpp

```
setcolor(RED);
setbkcolor(BLUE)
```

Drawing Lines

Use the line() function to draw a line from (x1,y1) to (x2,y2) with the current drawing color.

cpp

```
line(x1, y1, x2, y2);
```

Drawing Shapes

i. rectangle() draws a rectangle given the (left,top) and (right,bottom) coordinates
ii. circle() draws a circle centered at (x,y) with radius r
iii. arc() draws an arc centered at (x,y) from start angle to end angle
iv. bar() draws a filled rectangle

Example
cpp
```
rectangle(100, 100, 200, 200);
circle(300, 200, 50);
```

Simple Animation

To create a simple animation:

i. Clear the screen with cleardevice()

ii. Draw the next frame

iii. Pause with delay(milliseconds)

iv. Repeat

Example

cpp
```
while(1) {
  cleardevice();
  drawNextFrame();
  delay(50);
}
```

9.0 Ray Tracing

9.1 Definition

Ray tracing is a rendering technique used to generate realistic images by simulating the path of light through a scene. It works by casting rays from the camera out into the 3D scene and calculating the color of each pixel based on the objects the rays intersect.

The key steps in implementing ray tracing in C++ are:

i. **Initialize the scene**: Define the camera, lights, and objects with their positions, materials, and properties.

ii. **Cast rays**: For each pixel in the output image, cast a ray from the camera through that pixel into the scene. The direction of the ray depends on the camera parameters.

iii. **Intersect rays with objects:** Test each ray for intersections with the objects in the scene. Use the object's material properties to determine how the ray is affected (reflected, refracted, absorbed, etc).

iv. **Shade the pixel:** Accumulate the color contributions from the ray's interactions with objects. This may involve recursively casting secondary rays for reflections and refractions.

v. **Repeat for each pixel:** Repeat steps 2-4 for every pixel in the output image to generate the final rendered result

9.2 Applications of Ray Tracing

i. **Film and Television Visual Effects**: Ray tracing is widely used in the film and TV industry to create realistic visual effects and computer-generated imagery (CGI). It enables simulating complex lighting effects like reflections, refractions, soft shadows, and global illumination that are difficult to achieve with other rendering techniques.

ii. **Product Design and Visualization**: Ray tracing is used extensively in CAD/CAM/CAE software for product design, engineering, and virtual prototyping. It allows designers and engineers to create photorealistic visualizations of proposed products and designs before physical manufacturing.

iii. **Architecture and Lighting Design**: Ray tracing enables architects and lighting designers to simulate how light will interact with buildings, interiors, and outdoor spaces. It helps optimize lighting plans and ensure desired illumination levels and aesthetics.

iv. **Medical and Scientific Visualization**: Ray tracing is used in molecular graphics and medical imaging to visualize complex 3D structures like molecules, proteins, and anatomical models. The ability to simulate transparency and cutaway views is particularly useful.

v. **Video Games:** The recent introduction of hardware-accelerated real-time ray tracing has enabled its use in some video games to enhance lighting effects like reflections, shadows, and global illumination. However, ray tracing remains computationally expensive compared to traditional rasterization techniques.